Gene Prediction with Hidden Markov Models

Hidden Markov Models (HMM) are widely used in various fields of research: speech recognition, automatic natural language processing, handwriting recognition, and bioinformatics.

The 3 main problems associated to HMMs are:

1. Evaluation :
    - Problem: Compute the probability of observing the sequence given an HMM:
    - Solution: **Forward Algorithm**
2. Decoding:
    - Problem: find the sequence of states that maximizes the probability of observing the sequences.
    - Solution: **Viterbi Algorithm**
3. Training/Estimation:
    - Problem: Adjust the parameters of the HMM model to maximize the probability of generating the sequence of observations from the training data
    - Solution: **Forward-Backward Algorithm**

In this TME, we will apply the Viterbi algorithm to molecular biology data, in particular for the problem of gene prediction.

Some Biological background

In this small project, we will see how statistical models can be used to extract information from raw biological data. The goal will be to specify Hidden Markov Models which will allow to annotate the positions of the genes in the genome.

The genome, the carrier of genetic information, can be thought of as a long sequence of characters written in a 4-letter alphabet: `A`, `C`, `G` and `T`. Each letter of the genome is also called a base pair (or bp). It is now relatively inexpensive to sequence a genome (some direct to consumer company have offers as low as a few hundred euros for a human genome). However, we cannot understand, simply from the series of letters, how this information is used by the cell (a bit like having an instruction manual written in an unknown language, or a compiled code with no information on the machine).

An essential element is the gene, which after transcription and translation will produce proteins, the molecules responsible for much of the biochemical activity of cells.

if you do not know about transcription and translation, a short video about the basics: https://youtube.com/shorts/mSMjwxNK2EU?feature=share

Ameoba sisters page https://www.youtube.com/c/AmoebaSisters

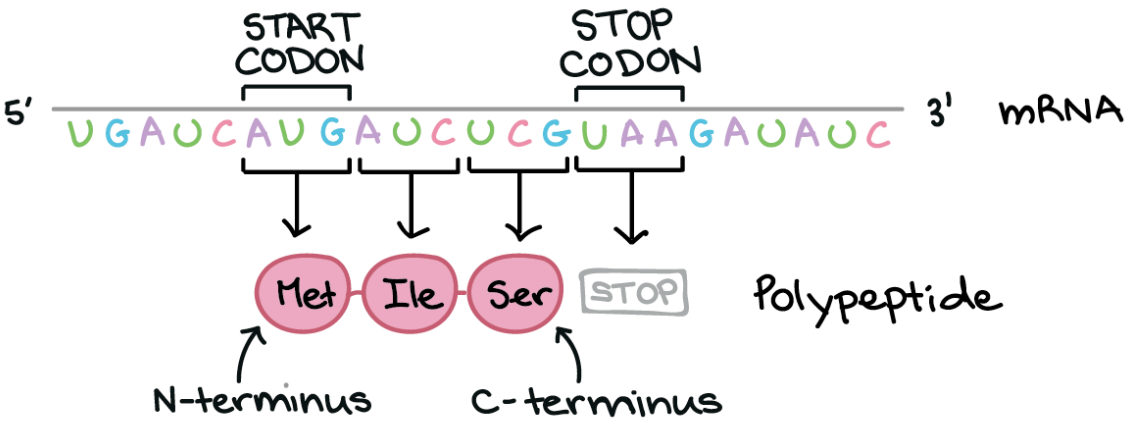
Two paragraphs summary for the impatient:

The translation into protein is done using the genetic code which, for each group of 3 letters (or bp) transcribed, matches an amino acid. These groups of 3 letters are called codons and there are 4^3, or 64. So, as a first approximation, a gene is defined by the following properties (for prokaryotic organisms):

- The first codon, called start codon is `ATG`,
- There are 61 codons which code for Amino Acids.
- The last codon, called the stop codon, marks the end of the gene and is one of the three sequences `TAA`, `TAG` or `TGA`. It does not appear in the gene.

We will integrate these different pieces of information to predict the positions of genes. Note that this figure is for the moment simplified, as we have omitted the fact that the DNA molecule consists of two complementary strands, and therefore that the genes present on the complementary strand are seen "upside down" on our sequence.

The regions between genes are simply called *intergenic regions*.



Important information to remember for the following: Each gene sequence starts with a start codon and ends with a stop codon

A few more biological information for the uninitiated

What is a chromosome? https://www.youtube.com/watch?v=IePMXxQ-KWY

Some more information about DNA and RNA (6min) https://youtu.be/JQByjprj_mA

And if you like to know how protein synthesis works (9min):
https://youtu.be/oefAI2x2CQM
And other video on the subject (3min): https://www.youtube.com/watch?v=gG7uCskUOrA

What is a gene (5min)?
https://www.youtube.com/watch?v=5MQdXjRPHmQ&list=PLInNVsmlBUlQT_peuWctrmGMiLngK-6fb&index=7

Here is a short (6min) video explaining the basis of gene regulation. Note that the part about operon is not important. https://youtu.be/h_1QLdtF8d0

Gene Modeling

Question 1: data download

We will be working on the first million bp of the E. coli genome (strain 042). Rather than working with the letters A, C, G, and T, we'll recode them with numbers ($A = 0$, $C = 1$, $G = 2$, $T = 3$).

The annotations provided are also encoded with integer values from 0 to 3:

- 0: the position is in a non-coding region = intergenic region
- 1: the position corresponds to a codon in phase 0
- 2: the position corresponds to a codon in phase 1
- 3: the position corresponds to a codon in phase 2

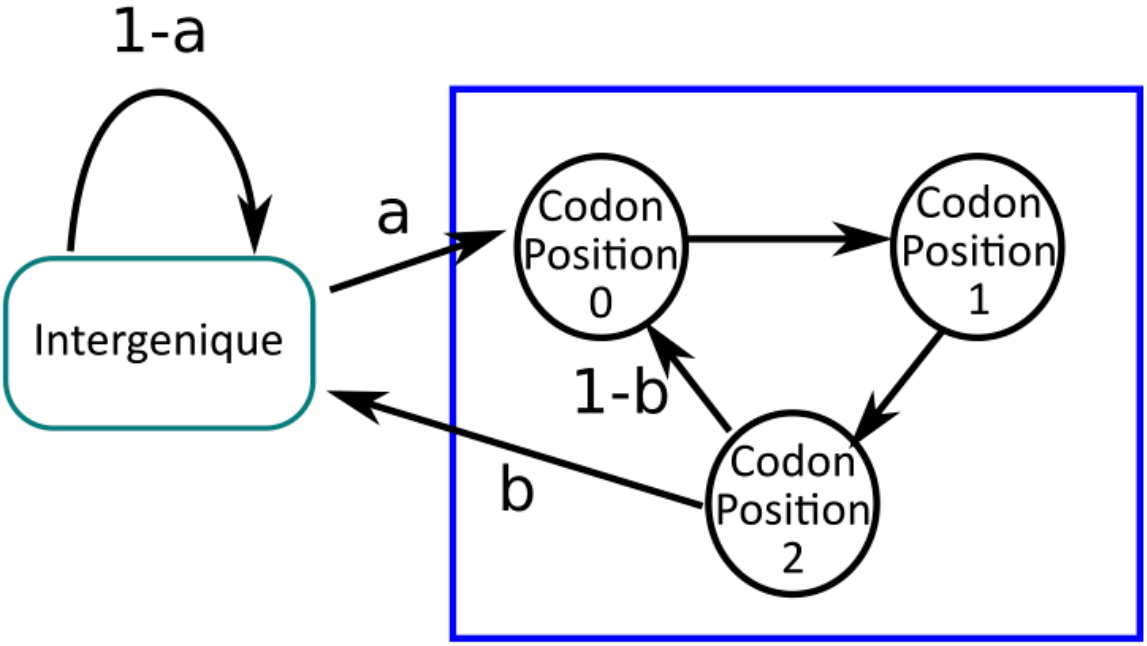
For instance for the drawing above we have the following values:

- `32031032031312300203031` for the sequence
- `00000123123123123000000` for the annotation

In [9]:
```
# Download pickle files for the genome sequences and
# its annotation
import numpy as np
import pickle as pkl

Genome=np.load('genome.npy') # the first Mio bp of E. coli
Annotation=np.load('annotation.npy')# gene annotation

## Let's split the data in two, one half for training and
## the second half for testing.

genome_train=Genome[:500000]
genome_test=Genome[500000:]

annotation_train=Annotation[:500000]
annotation_test=Annotation[500000:]
```

Question 2: Parameters Estimation / Learning

As the simplest model for separating codon sequences from intergenic sequences, we will define the hidden Markov chain whose transition graph is given below.



Such a model is defined as follows: we consider that there are 4 possible hidden states (intergenic, codon phase 0, codon phase 1, codon phase 2).

You can stay in the intergenic regions, and when you start a gene, the composition of each base of the codon is different. In order to be able to use this model to classify, it will be necessary to know the parameters for the transition matrix (so here only the probas a and b), and the distribution $(b_i, i = 0, \ldots, 3)$ of the nucleotides given the four states.

```
Pi = np.array ([1, 0, 0, 0]) ## we start in intergenic regions
A = np.array ([[1-a, a, 0, 0],
               [0, 0, 1, 0],
               [0, 0, 0, 1],
               [b, 1-b, 0, 0]])
B = ...
```

Given the structure of an HMM:

- The initial distribution Pi and the transition matrix A are estimated in the same way as for a simple Markov model (see lecture 4). In other words, the observations have no influence on the hidden states when they are known.
- The distribution of each observation only depends on the current state.

Given the nature of the data we use Multinoulli for the emissions. As a convenience we will store all the distributions b_i in a matrix B (emission probability matrix) structured as follows:

- K columns (number of possible states), N rows (number of states)
- Each row corresponds to an emission law for a state (ie, each row sums to 1)

We can now simply learn the parameters b_i with the two following steps:

1. for each state $i \in \Sigma$ store in cell $b_{i,j}$ the number of times the letter j was observed with state i.
2. Normalize the rows of B to sum to one.

Write the code of the function `def learnHMM (allX, allS, N, K):` which learns a model from the combined sequence of observations and sequence of states.

In [10]:
```
def learnHMM(allx, allq, N, K):
    """ Learn an HMM given a pair of observation and states
    np.array[int] * np.array[int] * int * int ->
            (np.array[double,double], np.array[double,double])
    return transition matrices A and B"""
    A = np.zeros((N, N))
    B = np.zeros((N, K))

    ### Your code here
    # Estimate transition counts
    for i in range(len(allq) - 1):
        A[allq[i], allq[i+1]] += 1

    # Estimate emission counts
    for i in range(len(allq)):
        B[allq[i], allx[i]] += 1

    # Normalize each row to turn counts into probabilities
    A = A / A.sum(axis=1, keepdims=True)
    B = B / B.sum(axis=1, keepdims=True)

    return A, B
```

In [11]:
```
Pi = np.array([1, 0, 0, 0])
nb_states= 4 ## (intergenic, codon 0, codon 1, codon 2)
nb_observation = 4 ## (A,C,G,T)
A,B =learnHMM(genome_train, annotation_train, nb_states, nb_observation)
print(A)
print(B)
```

```
[[0.99899016 0.00100984 0.         0.        ]
 [0.         0.         1.         0.        ]
 [0.         0.         0.         1.        ]
 [0.00272284 0.99727716 0.         0.        ]]
[[0.2434762  0.25247178 0.24800145 0.25605057]
 [0.24727716 0.23681872 0.34909315 0.16681097]
 [0.28462222 0.23058695 0.20782446 0.27696637]
 [0.1857911  0.26246354 0.29707437 0.25467098]]
```

You should find:

$A =$

```
[[0.99899016 0.00100984 0.         0.        ]
 [0.         0.         1.         0.        ]
 [0.         0.         0.         1.        ]
 [0.00272284 0.99727716 0.         0.        ]]
```

$B =$

```
[[0.2434762  0.25247178 0.24800145 0.25605057]
 [0.24727716 0.23681872 0.34909315 0.16681097]
 [0.28462222 0.23058695 0.20782446 0.27696637]
 [0.1857911  0.26246354 0.29707437 0.25467098]]
```

Note that each row sums to 1

Question 3: Decoding using Viterbi algorithm

It is not always easy to find the coding and non-coding regions of a genome. We would like to automatically annotate the genome, that is to say to find **the most probable sequence of hidden states** which made it possible to generate the observation sequence.

Reminders on the Viterbi algorithm (1967):

- It is used to estimate the most probable sequence of states given the observations and the model.
- It can be used to approximate the probability of observing the sequence given the model.

It uses two recursion variables:

probability: $\delta_i(t) = \log \max_{s_1^{t-1}} P(s_1^{t-1}, s_t = i, y_1^t)$

backtrack: $\Psi_j(t) = \arg\max_{i \in \Sigma} \delta_i(t-1) a_{ij}$

1. Initialization (indices starting at 0):

$$\begin{aligned} \delta_i(0) &= \log \pi_i + \log b_i(x_0) \\ \Psi_i(0) &= -1 \end{aligned}$$

Note: We initialize the first bactracking variable $\Psi_i(0)$ to -1 as this variable should not be used (-1 does not correspond to a state).

2. Recursion:

$$\begin{aligned} \delta_j(t) &= \left[\max_i \delta_i(t-1) + \log a_{ij}\right] + \log b_j(x_t) \\ \Psi_j(t) &= \arg\max_{i \in [1,\, N]} \delta_i(t-1) + \log a_{ij} \end{aligned}$$

3. Terminaison (with indices at $\{T-1\}$ in python)

$$S^\star = \max_i \delta_i(T-1)$$

4. Path

$$\begin{aligned} s^\star_{T-1} &= \arg\max_i \delta_i(T-1) \\ s^\star_t &= \Psi_{t+1}(s^\star_{t+1}) \end{aligned}$$

The estimate of $\log p(x_0^{T-1} \mid \lambda)$ is obtained by finding the greatest probability in the last column of δ.

Write down the algorithm of the `viterbi (x, Pi, A, B)` method:

Note: if you encounter problem with 0 cells giving infinite log values, you can try to add a very low value ϵ to all the cells of the transition matrix a_{ij} and for the emission

probabilities b_j (something like $\epsilon = 10^{-10}$). Be carefull to renormalize the rows of a and each of the b_j to sum to 1 afterward.

In [38]:
```
epsilon = 1e-10

A = A + epsilon
A = A / A.sum(axis=1, keepdims=True)

B = B + epsilon
B = B / B.sum(axis=1, keepdims=True)

Pi = Pi + epsilon
Pi = Pi / Pi.sum()

def viterbi(allx,Pi,A,B):
    """
    Parameters
    ----------
    allx : array (T,)
        Sequence d'observations.
    Pi: array, (K,)
        Distribution de probabilite initiale
    A : array (K, K)
        Matrice de transition
    B : array (K, M)
        Matrice d'emission matrix

    """
    ## initialisation
    psi = np.zeros((len(A), len(allx))) # A = N
    psi[:,0]= -1
    delta = np.zeros((len(A), len(allx))) #Initializing delta
    for i in range(len(A)):
        delta[i, 0] = np.log(Pi[i]) + np.log(B[i, allx[0]])

    ## recursion ...  (your code here)
    for t in range(1, len(allx)):
        for j in range(len(A)):
            max_prob = -np.inf
            max_state = 0
            for i in range(len(A)):
                prob = delta[i, t-1] + np.log(A[i, j])
                if prob > max_prob:
                    max_prob = prob
                    max_state = i
            delta[j, t] = max_prob + np.log(B[j, allx[t]])
            psi[j, t] = max_state
    return delta, psi

def get_path(psi, begin_q):
    """
    From the matrix of psi values and the value S*, get the path of maximal
    Parameters
    ----------
    psi: array (K,T)
    begin_q = int - value between 0 and K-1
    """
    ##Your code here
    T = psi.shape[1]
    path = np.zeros(T, dtype=int)
    path[T - 1] = begin_q
    for t in range(T - 2, -1, -1):
        path[t] = psi[path[t + 1], t + 1]
```

```
    return path
```

Here is a small sequence if you want to test your Viterbi code:

In [39]:
```
##Small code to test viterbi result
test_seq = "CGTGATATCATCAGGGCAGACCGGTTACATCCCCCTAACAAGCTGTTTAAAGAGAAATACTAT
dDNA = {"A": 0 , "C": 1 , "G": 2, "T": 3}
test_seqi = np.array([dDNA[c] for c in test_seq])

epsilon = 1e-10
A = A + epsilon
A = A / A.sum(axis=1, keepdims=True)

B = B + epsilon
B = B / B.sum(axis=1, keepdims=True)

viterbi(test_seqi, Pi, A, B)
delta, psi = viterbi(test_seqi, Pi, A, B)
last_state = np.argmax(delta[:, -1])
path = get_path(psi, last_state)
print(path)
```

```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2
 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3
 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 1
 2 3 1 2]
```

You should find the following state sequence after running Viterbi:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 1, 2, 3, 1, 2, 3,
1, 2,
       3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1,
2, 3,
       1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2,
3, 1,
       2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3,
1, 2,
       3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1,
2, 3,
       1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2,
3, 1,
       2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3,
1, 2,
       3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1,
2, 3,
       1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2,
3, 1,
       2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3,
1, 2,
       3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1,
```

```
2, 3,
       1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2])
```

You can now predict the states on your test sequences

In [40]:
```
delta, psi = viterbi(genome_test, Pi, A, B)
last_state = np.argmax(delta[:, -1])
predicted_states = get_path(psi, last_state)
#predicted_states = viterbi(genome_test, Pi, A, B)

test_annotation = annotation_test
```

Display

Lets simplify the annotation to two categories:

- **coding** (1)
- and **non coding** (0).

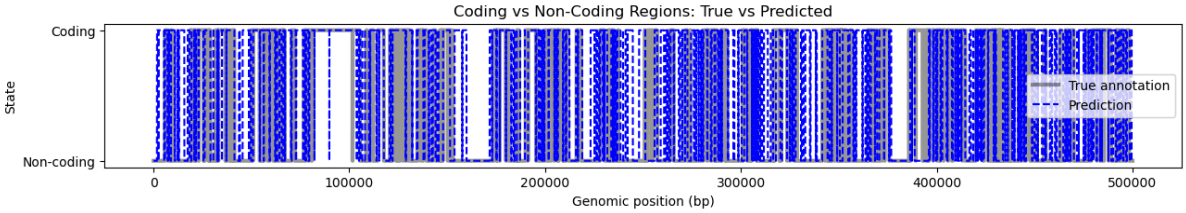
We can simply do that with a reallocation of the matrix of predictions (note that intergenic is state 0).

```
predicted_states[predicted_states != 0]=1
test_annotation[test_annotation != 0]=1
```

Then we will print for each genomic position if it is a coding or a non coding position using the true annotations and add the prediction on that.

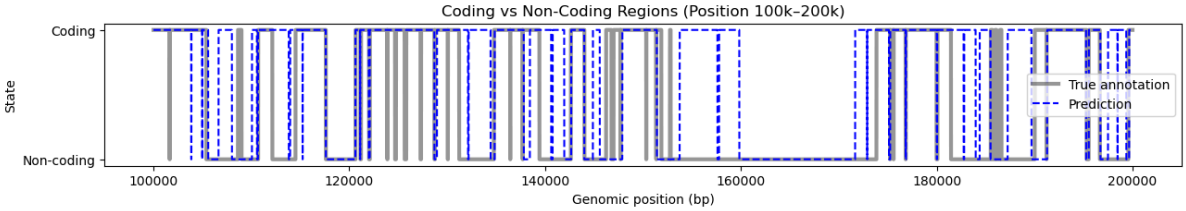
```
fig, ax = plt.subplots(figsize=(15,2))
ax.plot(test_annotation, label="annotation", lw=3, color="black",
alpha=.4)
ax.plot(predicted_states, label="prediction", ls="--")
plt.legend(loc="best")
plt.show()
```

In [41]:
```
##Display here

predicted_binary = predicted_states.copy()
true_binary = annotation_test[:len(predicted_states)].copy()

predicted_binary[predicted_binary != 0] = 1
true_binary[true_binary != 0] = 1

import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(15, 2))
ax.plot(true_binary, label="True annotation", lw=3, color="black", alpha=0.4
ax.plot(predicted_binary, label="Prediction", ls="--", color="blue")
plt.legend(loc="best")
plt.title("Coding vs Non-Coding Regions: True vs Predicted")
plt.xlabel("Genomic position (bp)")
plt.ylabel("State")
plt.yticks([0, 1], ["Non-coding", "Coding"])
plt.show()
```



You can consider a part of the genome, plot the values between the positions 100,000 and 200,000. Comment on the quality of the prediction.

In [43]:
```
start, end = 100_000, 200_000

fig, ax = plt.subplots(figsize=(15, 2))
ax.plot(range(start, end), true_binary[start:end], label="True annotation",
ax.plot(range(start, end), predicted_binary[start:end], label="Prediction",
plt.legend(loc="best")
plt.title("Coding vs Non-Coding Regions (Position 100k–200k)")
plt.xlabel("Genomic position (bp)")
plt.ylabel("State")
plt.yticks([0, 1], ["Non-coding", "Coding"])
plt.show()
```

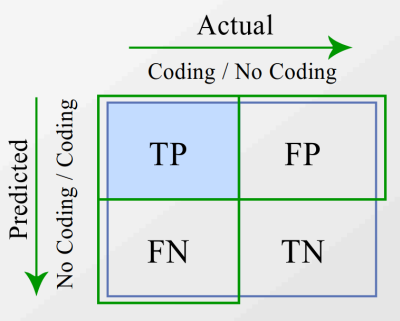


your comment here, which state are well predicted? Do we overpredict sometimes? Are there non coding regions predicted as coding? Why would the model predict that? Conversely, are there coding regions that are predicted as intergenic?

In the region between positions 100,000 and 200,000, the Viterbi prediction aligns reasonably well with the true annotation. Most coding regions are correctly identified, especially the longer ones with strong codon structure. The model successfully captures the expected periodic pattern of coding sequences. However, there are some false positives where non-coding regions are incorrectly predicted as coding. This typically happens in regions with a local base composition that resembles coding sequences—particularly when there is a high GC-content or repetitive patterns. There are also a few false negatives, where real coding regions are missed—especially near the edges of genes. Overall, the model performs well, but it sometimes overpredicts coding regions and may miss short or weakly expressed genes.

Question 4 : Performance Evaluation

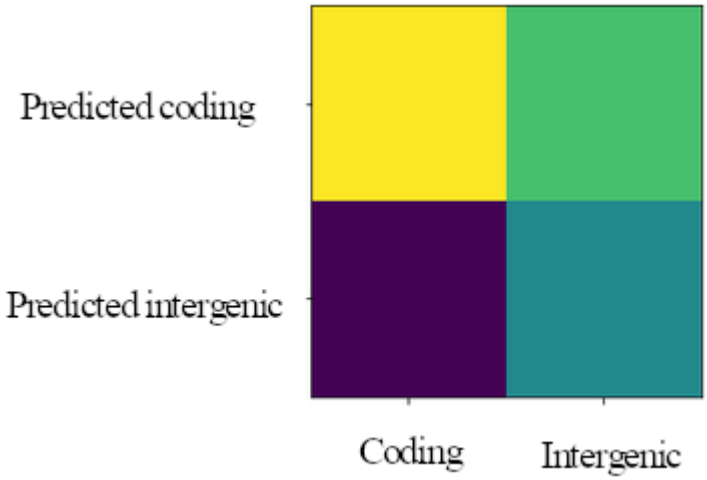
Using predictions and annotations of the genome, compute the confusion matrix.



In other words, we have:

- TP = True Positives, coding regions that are correctly predicted,
- FP = False Positives, intergenic regions predicted as coding regions,
- TN = True Negatives, intergenic regions correctly predicted,
- FN = False Negatives, coding regions predicted as intergenic.

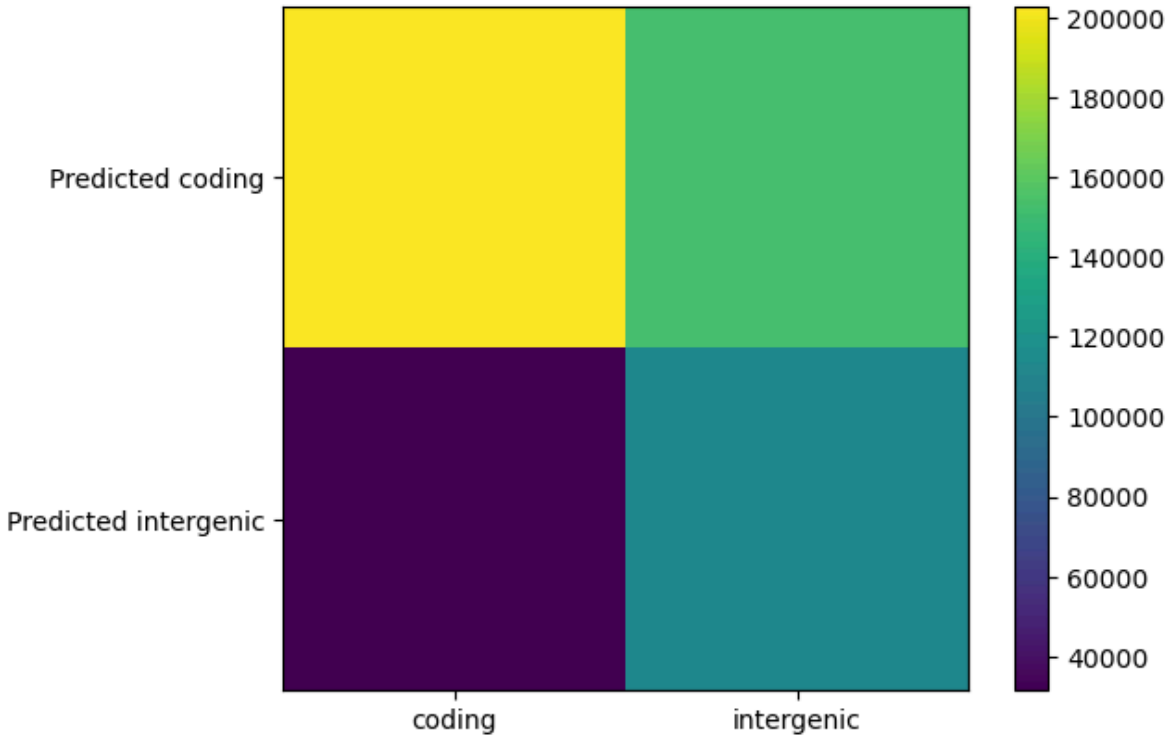
non coding state has index 0, the other states $(1, 2, 3)$ are **coding** states.



In [44]:
```
def create_confusion_matrix(true_sequence, predicted_sequence):
    ## your code here
    # Convert to binary: 0 = intergenic (non-coding), 1 = coding (1,2,3)
    true_binary = np.copy(true_sequence)
    predicted_binary = np.copy(predicted_sequence)

    true_binary[true_binary != 0] = 1
    predicted_binary[predicted_binary != 0] = 1

    # Compute confusion matrix values
    TP = np.sum((true_binary == 1) & (predicted_binary == 1))
    FP = np.sum((true_binary == 0) & (predicted_binary == 1))
    TN = np.sum((true_binary == 0) & (predicted_binary == 0))
    FN = np.sum((true_binary == 1) & (predicted_binary == 0))

    # Return confusion matrix in the format:
    # [[TP, FP],
    #  [FN, TN]]
    return np.array([[TP, FP],
                     [FN, TN]])

###Display the confusion matrix
import matplotlib.pyplot as plt

mat_conf=create_confusion_matrix(annotation_test, predicted_states)
```

```
plt.imshow(mat_conf)
plt.colorbar()
ax = plt.gca();

# Major ticks
ax.set_xticks(np.arange(0, 2, 1));
ax.set_yticks(np.arange(0, 2, 1));

# Labels for major ticks
ax.set_xticklabels(['coding','intergenic']);
ax.set_yticklabels(['Predicted coding','Predicted intergenic']);

print(mat_conf)
plt.show()
```

```
[[202819 152699]
 [ 31460 113022]]
```



Give an interpretation of the results, can we use this model to predict the position of the genes in the genome?

From the plot, the coding area that gets correctly predicted as coding is very high. The intergenic area that gets predicted as coding is also high. Coding area predicted intergenic is very low, and intergenic predicted intergenic is moderate. This shows the model has a very low False Negative, it almost never misses a coding region. But it often over-predicts coding in non-coding regions. We can use the model but with limitation because it has high false positive rate.

Question 5: Generating new sequences

Using the model estimated $\Theta = \{Pi, A, B\}$, specify a function `create_seq(N,Pi,A,B)` that, given a sequence length `N` would return:

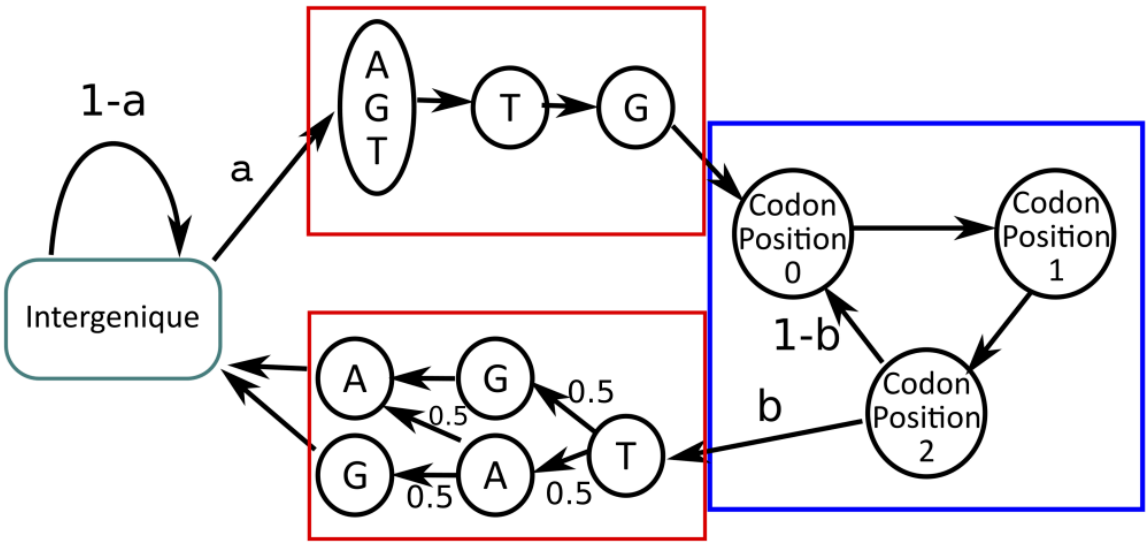
- a sequence of hidden states
- a sequence of observations.

In [45]:
```python
def create_seq(N,Pi,A,B):
    '''
    Return a sequence of N hidden states using Pi and A
    and for each hidden state return an observation using B
    '''
    ## your code here
    hidden_states = []
    observations = []

    # Sample initial state from Pi
    current_state = np.random.choice(len(Pi), p=Pi)
    hidden_states.append(current_state)

    # Sample first observation
    obs = np.random.choice(len(B[current_state]), p=B[current_state])
    observations.append(obs)

    for _ in range(1, N):
        # Transition to next state
        current_state = np.random.choice(len(A), p=A[current_state])
        hidden_states.append(current_state)

        # Generate observation from current state
        obs = np.random.choice(len(B[current_state]), p=B[current_state])
        observations.append(obs)

    return np.array(hidden_states), np.array(observations)
```

Question 6: Improving the model

Now let's assess if we can improve our prediction by incorporating an addtional layer of information in the model. We will take into account the gene boundaries by building a model that explicitly detects start codon and stop codon. We now want to integrate the additional information that says that a gene "always" begins with a start codon and "always" ends with a stop codon with the transition graph below.

The model now has 12 hidden states.



- Write the corresponding transition matrix, setting the transition probabilities between letters for stop codons to 0.5.

- Adapt the emissions matrix for all states of the model. You can reuse matrix B, calculated previously. The states corresponding to the stop codons will emit only one letter with a probability 1. For the start codon, we know that the proportions are as follows:
  - ATG : 83%,
  - GTG: 14%,
  - TTG: 3%

```
Pi2 = np.array(   [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ])  ##again,
we start in an intergenic region
A2 =  np.array([[1-a, a, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
                [0  , 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 ],
                  ... ])
B2 = ...
```

Assess the performances of the new model by comparing it to the first model on `genome_test`.

```
   predicted_states2=viterbi(genome_test,Pi2,A2,B2)
   predicted_states2[predicted_states2!=0]=1

fig, ax = plt.subplots(figsize=(15,2))
ax.plot(annotation_test, label="annotation", lw=3, color="black",
alpha=.4)
ax.plot(etat_predits, label="prediction model1", ls="--")
ax.plot(etat_predits2, label="prediction model2", ls="--")

plt.legend(loc="best")
plt.show()
```

Compute the confusion matrix with those new predictions and comment the results. Is it better than the previous one?

In [48]:
```
Pi2 = np.array(   [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 ])

# Transition matrix A2
a = 0.00100984  # transition to start codon
b = 0.00272284  # transition to stop codon

A2 = np.zeros((12, 12))

# State 0 and 10: intergenic
A2[0, 0] = 1 - a
A2[0, 1] = a * 0.83  # ATG
A2[0, 2] = a * 0.14  # GTG
A2[0, 3] = a * 0.03  # TTG
A2[10] = A2[0]

# Start codons go to coding phase 0
A2[1, 4] = 1
A2[2, 4] = 1
A2[3, 4] = 1

# Coding: 4 → 5 → 6
```

```
A2[4, 5] = 1
A2[5, 6] = 1
A2[6, 4] = 1 - b
A2[6, 7] = b / 3
A2[6, 8] = b / 3
A2[6, 9] = b / 3

# Stop codons go to intergenic (10)
A2[7, 10] = 1
A2[8, 10] = 1
A2[9, 10] = 1

# Dummy terminal state (not used)
A2[11, 11] = 1

B_trained = np.array([
    [0.2434762 , 0.25247178, 0.24800145, 0.25605057],  # intergenic
    [0.24727716, 0.23681872, 0.34909315, 0.16681097],  # codon phase 0
    [0.28462222, 0.23058695, 0.20782446, 0.27696637],  # codon phase 1
    [0.1857911 , 0.26246354, 0.29707437, 0.25467098],  # codon phase 2
])

# Build new B2 for 12 states
B2 = np.zeros((12, 4))

# State 0 & 10 (intergenic) -> same as B[0]
B2[0] = B_trained[0]
B2[10] = B_trained[0]

# States 1–3 (start codons) -> fixed emissions
# ATG: A, T, G → simulate by emitting A first
B2[1] = [1, 0, 0, 0]  # 'A'
B2[2] = [0, 0, 1, 0]  # 'G'
B2[3] = [0, 0, 0, 1]  # 'T'

# Coding phases (states 4,5,6) = B[1,2,3]
B2[4] = B_trained[1]
B2[5] = B_trained[2]
B2[6] = B_trained[3]

# Stop codons (TGA, TAG, TAA)
B2[7] = [0, 0, 1, 0]  # G (TGA → simulate by G)
B2[8] = [0, 0, 0, 1]  # T (TAG)
B2[9] = [1, 0, 0, 0]  # A (TAA)

# Final dummy state
B2[11] = [0.25, 0.25, 0.25, 0.25]  # uniform (not used)

A2, B2
```

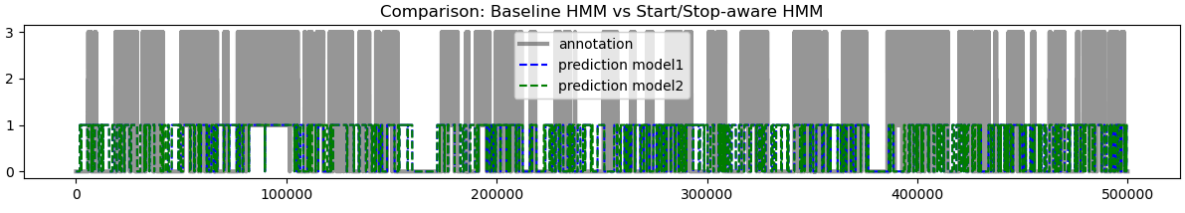
Out[48]:
```
(array([[9.98990160e-01, 8.38167200e-04, 1.41377600e-04, 3.02952000e-05,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         1.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         1.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         1.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 1.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 1.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         9.97277160e-01, 0.00000000e+00, 0.00000000e+00, 9.07613333e-04,
         9.07613333e-04, 9.07613333e-04, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
        [9.98990160e-01, 8.38167200e-04, 1.41377600e-04, 3.02952000e-05,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
         0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 1.00000000e+00]]),
 array([[0.2434762 , 0.25247178, 0.24800145, 0.25605057],
        [1.        , 0.        , 0.        , 0.        ],
        [0.        , 0.        , 1.        , 0.        ],
        [0.        , 0.        , 0.        , 1.        ],
        [0.24727716, 0.23681872, 0.34909315, 0.16681097],
        [0.28462222, 0.23058695, 0.20782446, 0.27696637],
        [0.1857911 , 0.26246354, 0.29707437, 0.25467098],
        [0.        , 0.        , 1.        , 0.        ],
        [0.        , 0.        , 0.        , 1.        ],
        [1.        , 0.        , 0.        , 0.        ],
        [0.2434762 , 0.25247178, 0.24800145, 0.25605057],
        [0.25      , 0.25      , 0.25      , 0.25      ]]))
```

In [52]:
```
epsilon = 1e-10

Pi2 = Pi2.astype(float)
Pi2 += epsilon
Pi2 = Pi2 / Pi2.sum()

A2 = A2.astype(float)
B2 = B2.astype(float)

A2 += epsilon
A2 = A2 / A2.sum(axis=1, keepdims=True)

B2 += epsilon
B2 = B2 / B2.sum(axis=1, keepdims=True)
```

```
genome_test_small = genome_test[:100000]  # or even 50,000
delta2, psi2 = viterbi(genome_test_small, Pi2, A2, B2)
delta2
```

Out[52]:
```
array([[-1.41273608e+00, -2.77612675e+00, -4.17145779e+00, ...,
        -1.36770902e+05, -1.36772266e+05, -1.36773643e+05],
       [-2.23327038e+01, -3.04242673e+01, -3.17876580e+01, ...,
        -1.36798536e+05, -1.36799914e+05, -1.36801277e+05],
       [-4.42599424e+01, -3.22040488e+01, -1.16402009e+01, ...,
        -1.36800316e+05, -1.36801694e+05, -1.36803057e+05],
       ...,
       [-2.23327038e+01, -4.52672134e+01, -4.66306040e+01, ...,
        -1.36793423e+05, -1.36806847e+05, -1.36807569e+05],
       [-2.37454398e+01, -2.36950841e+01, -2.47342954e+01, ...,
        -1.36783128e+05, -1.36789224e+05, -1.36786296e+05],
       [-2.37189981e+01, -2.47262691e+01, -2.60896598e+01, ...,
        -1.36787282e+05, -1.36788668e+05, -1.36790054e+05]])
```

In [53]:
```
epsilon = 1e-10

Pi2 = Pi2.astype(float)
Pi2 += epsilon
Pi2 = Pi2 / Pi2.sum()

A2 = A2.astype(float)
B2 = B2.astype(float)

A2 += epsilon
A2 = A2 / A2.sum(axis=1, keepdims=True)

B2 += epsilon
B2 = B2 / B2.sum(axis=1, keepdims=True)

# Run Viterbi for model 2
delta2, psi2 = viterbi(genome_test, Pi2, A2, B2)
etat_predits2 = get_path(psi2, np.argmax(delta2[:, -1]))
predicted_states2 = etat_predits2.copy()
predicted_states2[predicted_states2 != 0] = 1

# Ensure etat_predits from model 1 is binary
etat_predits = predicted_states.copy()
etat_predits[etat_predits != 0] = 1

# Plot both models
fig, ax = plt.subplots(figsize=(15, 2))
ax.plot(annotation_test, label="annotation", lw=3, color="black", alpha=.4)
ax.plot(etat_predits, label="prediction model1", ls="--", color="blue")
ax.plot(predicted_states2, label="prediction model2", ls="--", color="green"
plt.legend(loc="best")
plt.title("Comparison: Baseline HMM vs Start/Stop-aware HMM")
plt.show()
```



In [54]:
```
from sklearn.metrics import confusion_matrix, precision_score, recall_score,

def binarize_states(states):
    binary = np.copy(states)
```

```
    binary[binary != 0] = 1
    return binary

# Binarize true annotations and predictions
true_bin = binarize_states(annotation_test)
pred1_bin = binarize_states(etat_predits)        # model 1
pred2_bin = binarize_states(predicted_states2)   # model 2

# Compute confusion matrices
cm1 = confusion_matrix(true_bin, pred1_bin)
cm2 = confusion_matrix(true_bin, pred2_bin)

# Print confusion matrices
print("Confusion Matrix - Model 1 (Baseline HMM):")
print(cm1)
print("\nConfusion Matrix - Model 2 (Start/Stop-aware HMM):")
print(cm2)

# Compute performance metrics
def print_metrics(y_true, y_pred, model_name=""):
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)
    print(f"\n📊 Performance for {model_name}")
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall   : {rec:.4f}")
    print(f"F1 Score : {f1:.4f}")

# Print metrics for both models
print_metrics(true_bin, pred1_bin, model_name="Model 1 (Baseline HMM)")
print_metrics(true_bin, pred2_bin, model_name="Model 2 (Start/Stop-aware HMM
```

```
/Users/demimao/opt/anaconda3/lib/python3.9/site-packages/pandas/core/array
s/masked.py:60: UserWarning: Pandas requires version '1.3.6' or newer of 'b
ottleneck' (version '1.3.5' currently installed).
  from pandas.core import (
Confusion Matrix - Model 1 (Baseline HMM):
[[113022 152699]
 [ 31460 202819]]

Confusion Matrix - Model 2 (Start/Stop-aware HMM):
[[107569 158152]
 [ 29499 204780]]

📊 Performance for Model 1 (Baseline HMM)
Accuracy : 0.6317
Precision: 0.5705
Recall   : 0.8657
F1 Score : 0.6878

📊 Performance for Model 2 (Start/Stop-aware HMM)
Accuracy : 0.6247
Precision: 0.5642
Recall   : 0.8741
F1 Score : 0.6858
```

Comment: Overall, model 2 improves recall slightly, but also increases false positives. Model 2 has slightly lower accuracy and precision, so model 2 doesn't outperform model 1.

Question 7: Integrating reverse strand

We now want to add the information about the genes that could come from the complementary strand.

If you are unsure about what the forward and complementary strands means, you can check the videos above and the following video that summarizes it (the end about gene order is not important): https://youtu.be/JC6ew2xnJBA

In summary for a sequence of DNA that is written as `GCGATGCGTTGTAAACGCGATCAGCGCAT`, we have in fact two sequences, one for each strand:

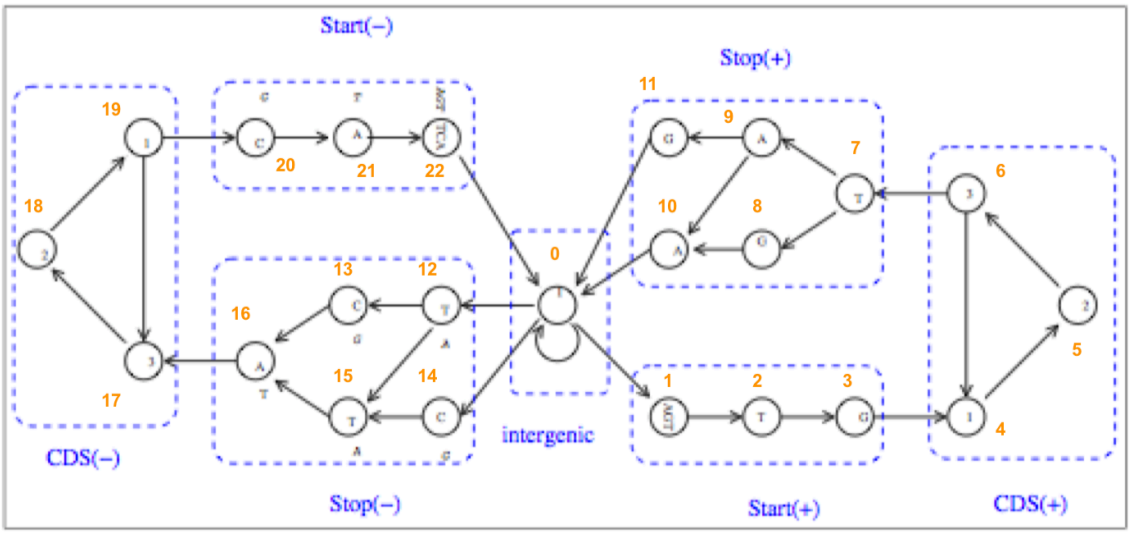
```
                x--------->
          5'  GCGATGCGTTGTAAACGCGATCAGCGCATGGG  3'   forward
(or plus) strand
            ||||||||||||||||||||||||||||||||
          3'  CGCTACGCAACATTTGCGCTAGTCGCGTACCC  5'
complementary (or minus) strand
                                <-------x
```

On the example above, there are two genes, one on the forward strand and one on the complementary strand. Because the genes are annotated using only forward strand, we will need to detect the gene information using the *reverse complementary sequence*.

In other words, we will add states for genes on the minus strand in the following way:

- we enter a minus gene with either `TCA`, `CTA`, `TTA` (reverse complementary of the stop codons: `TGA`, `TAG`, `TAA`)
- we leave a gene with `[C]A[TCA]` (reverse complementary of a start codon)

The corresponding transition graph is as follow (You can see that this model has 22 states, numbered in orange):



A perfect annotation for our small example sequence woul be:

```
                    x---------->
          5'  GCGATGCGTTGATAAACGCGATCAGCGCATGGG  3'   forward
(or plus) strand
              ||||||||||||||||||||||||||||||||
          3'  CGCTACGCAACTATTTGCGCTAGTCGCGTACCC  5'
complementary (or minus) strand
                                    <-------x
                              1     111111222          state
number
              000123456456790000000236789012000
```

To implement such a model, you will have to

- deduce from the observation matrices for codons a second one for the codons that are seen on the reverse strand. You can make the hypothesis that the codon distribution is the same on both strands.
- encode the transition matrix for observing genes on the reverse strand, starting from a reverse codon stop and ending with a codon start.

Implement a third model `Pi3, A3, B3` that would take into account the reverse strand and evaluate its performances with respect to model 1 and model 2.

Note: Be careful with the evaluation, the annotation given only provides the genes that are on the forward strand (the genes on the reverse strand are annotated as intergenic). If we use the numbering provided in the figure, we would do something like:

```
predicted_states[predicted_states > 11]=0 #reverse strand genes as
negatives
predicted_states[predicted_states != 0]=1 #forward strand genes as
positives
```

In [55]:
```
Pi3 = np.zeros(22)
Pi3[0] = 1  # Start in forward intergenic
A3 = np.zeros((22, 22))
a = 0.00100984  # entry into start codon
b = 0.00272284  # exit into stop codon

# --- Forward strand (states 0–10) ---
A3[0, 0] = 1 - a
A3[0, 1] = a * 0.83  # ATG
A3[0, 2] = a * 0.14  # GTG
A3[0, 3] = a * 0.03  # TTG
A3[10] = A3[0]       # forward intergenic after stop

A3[1, 4] = 1
A3[2, 4] = 1
A3[3, 4] = 1
A3[4, 5] = 1
A3[5, 6] = 1
A3[6, 4] = 1 - b
A3[6, 7] = b / 3
A3[6, 8] = b / 3
A3[6, 9] = b / 3
A3[7, 10] = 1
A3[8, 10] = 1
A3[9, 10] = 1

# --- Reverse strand (states 11–21) ---
```

```
A3[11, 11] = 1 - a
A3[11, 12] = a / 3  # TCA
A3[11, 13] = a / 3  # CTA
A3[11, 14] = a / 3  # TTA

A3[12, 15] = 1
A3[13, 15] = 1
A3[14, 15] = 1
A3[15, 16] = 1
A3[16, 17] = 1
A3[17, 15] = 1 - b
A3[17, 18] = b / 3  # CAT
A3[17, 19] = b / 3  # CAC
A3[17, 20] = b / 3  # CAA
A3[18, 21] = 1
A3[19, 21] = 1
A3[20, 21] = 1
A3[21] = A3[11]     # return to reverse intergenic

B3 = np.zeros((22, 4))

# Reuse B_trained for coding/intergenic phases
B_intergenic = [0.2434762 , 0.25247178, 0.24800145, 0.25605057]
B_coding_0   = [0.24727716, 0.23681872, 0.34909315, 0.16681097]
B_coding_1   = [0.28462222, 0.23058695, 0.20782446, 0.27696637]
B_coding_2   = [0.1857911 , 0.26246354, 0.29707437, 0.25467098]

# --- Forward strand ---
B3[0] = B_intergenic
B3[1] = [1, 0, 0, 0]  # ATG → A
B3[2] = [0, 0, 1, 0]  # GTG → G
B3[3] = [0, 0, 0, 1]  # TTG → T
B3[4] = B_coding_0
B3[5] = B_coding_1
B3[6] = B_coding_2
B3[7] = [0, 0, 1, 0]  # TGA → G
B3[8] = [0, 0, 0, 1]  # TAG → T
B3[9] = [1, 0, 0, 0]  # TAA → A
B3[10] = B_intergenic

# --- Reverse strand ---
B3[11] = B_intergenic
B3[12] = [1, 0, 0, 0]  # TCA → A
B3[13] = [1, 0, 0, 0]  # CTA → A
B3[14] = [1, 0, 0, 0]  # TTA → A
B3[15] = B_coding_0
B3[16] = B_coding_1
B3[17] = B_coding_2
B3[18] = [0, 1, 0, 0]  # CAT → C
B3[19] = [0, 1, 0, 0]  # CAC → C
B3[20] = [0, 1, 0, 0]  # CAA → C
B3[21] = B_intergenic

# Normalize B3 to avoid log(0)
epsilon = 1e-10
B3 += epsilon
B3 = B3 / B3.sum(axis=1, keepdims=True)
```

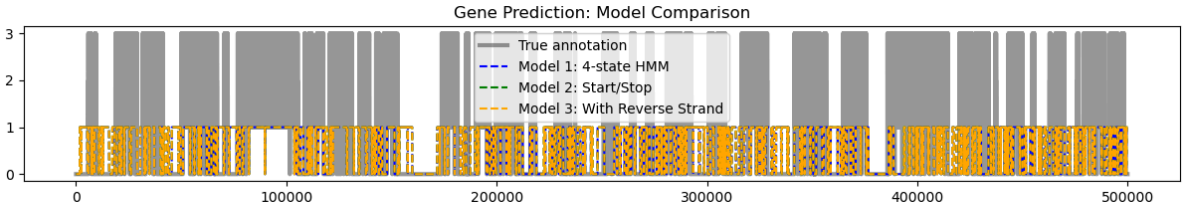
In [57]:

```
epsilon = 1e-10

Pi3 = Pi3.astype(float)
Pi3 += epsilon
Pi3 = Pi3 / Pi3.sum()
```

```
A3 = A3.astype(float)
B3 = B3.astype(float)

A3 += epsilon
A3 = A3 / A3.sum(axis=1, keepdims=True)

B3 += epsilon
B3 = B3 / B3.sum(axis=1, keepdims=True)

delta3, psi3 = viterbi(genome_test, Pi3, A3, B3)
etat_predits3 = get_path(psi3, np.argmax(delta3[:, -1]))

# Keep only forward strand predictions (states 1–10) as coding
etat_predits3[etat_predits3 > 10] = 0
etat_predits3[etat_predits3 != 0] = 1
```

In [58]:

```
import matplotlib.pyplot as plt

# Make sure all predictions are binary
etat_predits1 = np.copy(etat_predits)          # from model 1
etat_predits1[etat_predits1 != 0] = 1

etat_predits2 = np.copy(predicted_states2)     # from model 2
etat_predits2[etat_predits2 != 0] = 1

etat_predits3 = np.copy(etat_predits3)         # from model 3 (already binari

# Plot
fig, ax = plt.subplots(figsize=(15, 2))
ax.plot(annotation_test, label="True annotation", color="black", lw=3, alpha
ax.plot(etat_predits1, label="Model 1: 4-state HMM", ls="--", color="blue")
ax.plot(etat_predits2, label="Model 2: Start/Stop", ls="--", color="green")
ax.plot(etat_predits3, label="Model 3: With Reverse Strand", ls="--", color=
plt.legend(loc="best")
plt.title("Gene Prediction: Model Comparison")
plt.show()
```



In [59]:

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_scor

def evaluate_model(y_true, y_pred, model_name=""):
    cm = confusion_matrix(y_true, y_pred)
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    print(f"\n {model_name}")
    print("Confusion Matrix:\n", cm)
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall   : {rec:.4f}")
    print(f"F1 Score : {f1:.4f}")
    return cm

# True labels (binarized)
annotation_binary = np.copy(annotation_test)
```

```
annotation_binary[annotation_binary != 0] = 1

# Evaluate all models
cm1 = evaluate_model(annotation_binary, etat_predits1, "Model 1: 4-state HMM
cm2 = evaluate_model(annotation_binary, etat_predits2, "Model 2: Start/Stop-
cm3 = evaluate_model(annotation_binary, etat_predits3, "Model 3: Start/Stop
```

```
📊 Model 1: 4-state HMM
Confusion Matrix:
 [[113022 152699]
 [ 31460 202819]]
Accuracy : 0.6317
Precision: 0.5705
Recall   : 0.8657
F1 Score : 0.6878

📊 Model 2: Start/Stop-aware HMM
Confusion Matrix:
 [[107569 158152]
 [ 29499 204780]]
Accuracy : 0.6247
Precision: 0.5642
Recall   : 0.8741
F1 Score : 0.6858

📊 Model 3: Start/Stop + Reverse Strand
Confusion Matrix:
 [[107569 158152]
 [ 29499 204780]]
Accuracy : 0.6247
Precision: 0.5642
Recall   : 0.8741
F1 Score : 0.6858
```

Comments:

Model 1 is a bit more conservative (fewer false positives). Model 2/3 detect slightly more true genes (higher recall), but at the cost of more overprediction. For model 3, if reverse strand annotations were available, we would get additional predictive power.

Question 8: Implementing the forward-backward algorithm (optional)

Using the information presented in the lecture, implement an EM estimation of the parameter based on the forward-backward algorithm.

1. Write a function for the forward algorithm
2. Write a function for the backward algorithm
3. Deduce a function to compute the smoothing probabilities
4. Write the EM algorithm and compare the estimation results with the viterbi based method.

Did you encounted any problem while implementing this algorithm? Detail and comment each step of your analysis

Note: The forward and backward values decrease exponentially fast with the length of the sequence, leading to numerical issues. **You will need to integrate rescaling factors**

for the probabilities (as described in the lecture).

In [ ]: